

CodeOrigin

Does adding repository history to an LLM-RAG pipeline improve its answers? Evaluation report, organised by the grading criteria.

Repository: github.com/IATESPAGHETTI/Code_Origin | Held-out run: #8 on [pallets/itsdangerous](#) | Dataset hash [b68e5bd36697c3ba](#)

In one paragraph. CodeOrigin is a repository-aware RAG system that answers "why was it built this way?" from a codebase and its history (commits, diffs, issues, pull requests, reviews), with citations, and refuses when the history does not say. We tested whether adding history helps by asking the same questions in four modes (no context, code only, code + history, oracle) with everything else fixed, and judging the result with a rule written before running. On a held-out real repository ([pallets/itsdangerous](#): 677 commits, 125 issues, 311 PRs, history written by other people), adding history raised correctness on history questions by **+0.32 (llama2)** and **+0.28 (codellama:7b)**, with 95% intervals that exclude 0, so both pass the rule. A 3B model (starcoder2) showed no effect. An independent LLM judge confirms the result. Weaknesses we found are reported below.

Criterion	Where it is answered	Status
1. Research question and alignment	Section 1	Done
2. Quantitative evaluation	Section 2	Done; human validation partial
3. Results and interpretation	Section 3	Done
4. Technical implementation	Section 4	Done
5. Limitations and failure analysis	Section 5	Done
6. Methodology and reproducibility	Section 6	Done
7. Individual understanding	Section 7	Summary here; to be defended in person

1. Research question and experimental alignment

Criterion: clarity of the question, alignment between problem, objectives, methodology and experiment, and whether the evaluation can answer the question.

Research question. Does adding repository history (commits, diffs, issues, PRs, reviews) to an LLM-RAG pipeline improve the accuracy and usefulness of answers to "why was it built this way?" questions?

Problem	Objective	Method	Experiment
Assistants see only current code, so "why" questions become guesses; the reasons live in history.	Show whether retrieving history improves answers, for which models and which question types.	Hybrid RAG over code plus linked history, with citations and refusal when evidence is missing.	Same questions in four modes; paired comparison of code_history vs code_only on history questions; decision rule fixed in advance.

Mode	What the model sees	Why it exists
no_context	Question only	What the model guesses or already knows
code_only	Question + current code and docs	Realistic baseline (how most tools work)
code_history	Code + commits, diffs, issues, PRs, reviews	The treatment (the hypothesis)
oracle	Question + gold evidence, no retrieval	Separates retrieval failure from the model failing to use evidence

Decision rule (fixed before any run). History is "useful" only if code_history beats code_only on history questions by at least **0.10** correctness, the 95% bootstrap interval **excludes 0**, and the control category (questions answerable from current code) loses no more than **0.05**. There is no blended score: results are per category.

Why the experiment can answer the question. (a) It is paired: every question is asked under every mode, so differences come only from the evidence shown (same prompt shape, 6000-character context budget, temperature 0, seed 42). (b) The control category checks that history does not hurt questions that do not need it.

(c) The oracle mode isolates retrieval from comprehension. (d) The headline test is on a held-out real repository whose history was written by other people; the demo repository (written by us) was used only to build and tune, so its circularity cannot inflate the headline result.

2. Quantitative evaluation

Criterion: appropriate metrics, evaluation data, baselines and measurable evidence.

Dataset	Items	Role
Demo repository "Ledgerly" (fictional, deterministic)	43 (23 dev, 20 test)	Build and tune; thresholds calibrated on dev only (run #5 on dev)
pallets/itsdangerous (real, snapshot at commit 672971d)	35, all held-out: 24 history, 4 control, 7 guardrail	Headline test (run #8, 399 answers); dataset committed before the run

Baselines: no_context and code_only. **Models:** llama2, codellama:7b, starcoder2:3b (all local via Ollama); gemma:2b as the in-service jury.

Metrics (per answer). Correctness = fraction of key facts present (alternative phrasings; for questions that must be refused, 1 if the system refused or abstained). Evidence recall and precision (was the gold evidence retrieved). Citation validity and whether the gold source was cited. Unsupported-sentence ratio (lexical grounding check). Refusal correctness, latency, tokens. A failure tag for each wrong answer (retrieval_miss, model_ignored_evidence, hallucinated, over_refused, under_refused).

Statistics. Per category mean with 95% bootstrap CI (2000 resamples); paired comparisons with bootstrap CI, a paired sign-flip permutation test and Cohen's d_z; quadratic-weighted kappa and Spearman correlation for scorer agreement.

Validation of the scoring method

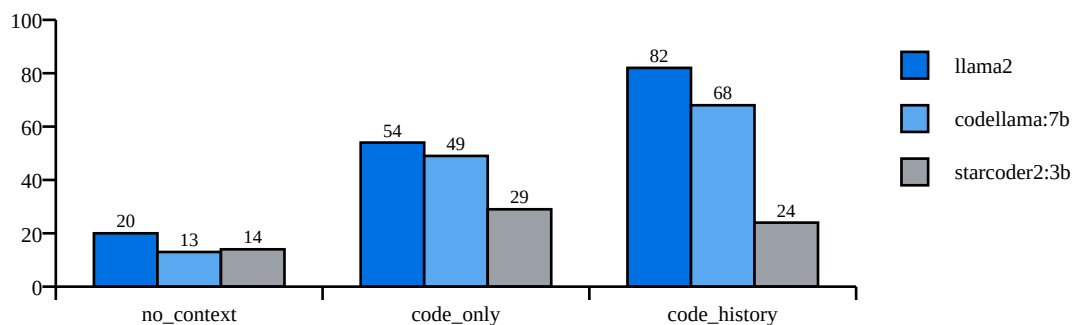
Check	Sample	Result
DeepSeek LLM judge (blind to model and mode)	all 399 answers	kappa 0.75, exact agreement 75%
GPT LLM judge	36 sampled answers	kappa 0.94 (all scores in the sample were 0 or full, so the easy case)
Gemini LLM judge	same 36 answers	kappa 0.69; judges agree with each other at 0.80 to 0.86
Author hand check (not a domain expert)	18 of the 36: the rows checkable by comparing text	18 of 18 agree with the automatic score; the 16 technical rationale rows are not human-graded

The automatic score is somewhat generous: of 143 answers it scored fully correct, DeepSeek rated 42 (29%) as only partly right or wrong; of 204 it scored 0, the judge gave credit to 24 (12%). The judges are LLMs, so this is not human grading. The earlier in-service gemma:2b jury gave every answer 100% and is not used.

3. Results and interpretation

Criterion: presentation and interpretation, comparison with the baseline, patterns, and justified conclusions.

Overall correctness (%) by mode, held-out run #8



Pre-registered test: code_history minus code_only, history questions

Model	Repository	Effect	95% CI	p (permutation)	Verdict by the rule
llama2	Real (held-out), n=24	+0.32	[+0.07, +0.55]	0.022	History is useful
codellama:7b	Real (held-out), n=24	+0.28	[+0.05, +0.49]	0.033	History is useful
starcoder2:3b	Real (held-out), n=24	-0.02	[-0.21, +0.17]	1.00	No effect detected
llama2	Demo (dev), n=13	+0.42	[+0.15, +0.69]	0.023	History is useful
codellama:7b	Demo (dev), n=13	+0.42	[0.00, +0.73]	0.087	Inconclusive (interval touches 0)
starcoder2:3b	Demo (dev), n=13	+0.08	[0.00, +0.19]	0.50	Inconclusive

Does it survive a different scorer? Re-scoring the whole comparison with the DeepSeek judge (scores rescaled to 0-1): llama2 +0.35 [+0.15, +0.54], codellama +0.35 [+0.12, +0.58], starcoder2 +0.06 [-0.08, +0.23]. Same conclusions. Excluding the two items an independent review flagged (it06, it35) gives +0.33 and +0.27 for the 7B models, with the same verdicts.

Patterns and interpretation

- **History helps capable models, and the mechanism is retrieval.** Evidence recall rises from 0.14 (code_only) to 0.70 (code_history); correctness follows. Mean judge score confirms the ordering no_context < code_only < code_history for both 7B models.
- **Largest gains are on look-up style history questions:** issue linkage 0.50 to 1.00 and change attribution 0.00 to 1.00 for both 7B models; design rationale 0.35 to 0.70 (codellama) and 0.45 to 0.75 (llama2).
- **Same effect size, different verdict.** On the demo, codellama had the same +0.42 as llama2 but only 13 questions, so its interval touched 0. That means not enough evidence, not no effect; with 24 questions on the real repository it passes.
- **Model size matters.** starcoder2:3b gets no benefit on either repository: even with perfect evidence (oracle) 18 of its 28 answers were tagged hallucinated. Its weakness is using evidence, not finding it.
- **The effect is smaller on real history** (+0.3 vs +0.4 on the demo), as expected when retrieval is harder (recall 0.70, not 1.00) and the history was not written to fit the questions.
- **Conclusion we claim:** supported on one real repository for 7B models. Not claimed: that it generalises across repositories, languages or larger models.

Which model is best for which task?

Each task is mapped to a question category of the real-repository dataset. Values are mean correctness in the RAG mode (code_history): automatic score, then the DeepSeek judge in brackets. With 2 to 10 questions per task, a gap under 0.10 is called a tie.

Task (question category)	n	llama2	codellama:7b	starcoder2:3b	Best
Explanation: why was it built this way (design_rationale)	10	0.75 (0.50)	0.70 (0.45)	0.15 (0.25)	llama2 = codellama (tie)
Bug analysis (bug_origin)	6	0.69 (0.58)	0.61 (0.58)	0.17 (0.00)	tie of the 7B models
Dependency understanding: links between issues, PRs, commits (issue_linkage)	4	1.00 (1.00)	1.00 (1.00)	0.50 (0.25)	tie of the 7B models
Code retrieval: look up a fact in current code (current_state)	4	0.75 (0.38)	1.00 (0.88)	0.25 (0.25)	codellama:7b
RAG, whole pipeline: no_context / code_only / code_history (28 answerable questions)	28	0.21 / 0.50 / 0.77	0.12 / 0.50 / 0.74	0.14 / 0.25 / 0.20	llama2 = codellama on accuracy
Code generation	-	not evaluated	not evaluated	not evaluated	-
Refactoring	-	not evaluated	not evaluated	not evaluated	-

Not evaluated, and why. CodeOrigin answers questions about a repository and its history; it does not generate or modify code. The dataset therefore has no code-generation or refactoring tasks and there is no code test-pass

rate. Evaluating them would need a different dataset (tasks with unit tests that generated code must pass); they are listed as future work, not guessed at.

Metrics covered (code_history, all 35 questions pooled)

Metric	llama2	codellama:7b	starcoder2:3b
Accuracy / correctness (answerable questions)	0.77	0.74	0.20
Hallucination rate (answers with at least half the sentences unsupported; refusals excluded)	31%	22%	59%
Unsupported-sentence ratio (mean)	0.27	0.26	0.37
Response latency, median (mean)	8.3 s (12.0 s)	16.1 s (14.5 s)	1.4 s (2.3 s)
Tokens in (median) / out, median (mean)	2217 / 90 (126)	2217 / 193 (173)	1893 / 44 (147)
Relevance	not scored separately	not scored separately	not scored separately

Retrieval quality belongs to the retriever, so it is identical for all models: evidence recall / precision 0.14 / 0.05 (code_only), 0.70 / 0.25 (code_history), 1.00 / 1.00 (oracle). Resource use was not profiled beyond latency and token counts (one 6 GB GPU).

Reading it. For every task history can help (explanation, bug analysis, dependency links) llama2 and codellama:7b are indistinguishable at this sample size and starcoder2:3b is clearly worse. The only clear winner is current-code lookup, where codellama:7b leads (4 questions, weak evidence). The practical difference between the two 7B models is cost: llama2 is faster at the median (8.3 s vs 16.1 s) but the means are closer (12.0 s vs 14.5 s), and codellama hallucinated less (22% vs 31%). starcoder2:3b is fastest (1.4 s median) but hallucinated in 59% of answers. These metrics are also exported to Prometheus and shown in Grafana.

4. Technical implementation

Criterion: correctness and completeness of the AI/LLM workflow, tools, models, retrieval and automation.

Architecture. Seven containers behind one gateway: gateway (limits, rate limit, API key, metrics), ingest (git + GitHub API, SQLite), rag (hybrid retrieval), llm (Ollama client + jury), orchestrator (pipeline + guardrails), eval (runs, statistics, reports), ChromaDB, plus the React/Tailwind web app. Ollama runs on the host GPU (6 GB).

Retrieval. Hybrid search: vector similarity (all-MiniLM-L6-v2 embeddings in ChromaDB, cosine) plus BM25 keyword search, fused with reciprocal-rank fusion, so exact SHAs, issue numbers and identifiers still match. Commits, diffs, issues, PRs and reviews are cross-linked (#123 mentions, PR merge commits) so retrieving one pulls in the others that explain it. Chunk IDs are deterministic, so re-ingesting never duplicates; secrets are redacted before indexing.

Generation and guardrails (enforced in code, not by asking the model to behave): G1 off-topic refused without a model call; G2 insufficient-history abstention; G3 invented citations stripped; G4 lexical grounding check; G5 prompt-injection defence (commit text is untrusted data); G6 secret redaction at ingest and output; G7 input limits, rate limit, API key; G8 repository isolation; G9 signed webhooks; G10 refusal of requests for secrets or commands.

Evaluation harness. Two-pass runner (generate all answers, then judge), stored per-answer metrics and full responses, bootstrap and permutation statistics, blind grading-sheet export, kappa tooling, a live progress view and a results page on the website ("which model is best where").

DevOps. Multi-stage non-root images with health checks and read-only filesystems; Docker Compose with dev, prod, monitoring and demo overrides; GitHub Actions CI with 11 jobs (lint, drift checks, six test jobs, frontend build, gitleaks + Trivy security scan, compose smoke test, end-to-end eval gate); a release workflow that scans and publishes images to GHCR on a version tag; Prometheus, Grafana and Loki run with the demo stack (details and screenshots in the appendix). 142 backend and 20 frontend tests. Running the pipeline for real found and fixed a HIGH dependency vulnerability, root-user Dockerfiles, a linter version drift and a wrong script call.

5. Limitations and failure analysis

Criterion: understanding of what works and what does not, failure cases, assumptions, and sources of error or bias.

Failure analysis (real cases from the held-out run)

Failure	What happened	Response / status
Retrieval misses unusual phrasing	"Why was 1.0.0 removed from PyPI?" retrieved unrelated issues (#92, #47); recall 0 for that item. Both 7B models invented a reason, and the evolution category (n=2) got worse with history.	Query rewriting; not done
History can distract on code-only questions	With history in the prompt llama2 said the default key derivation is "hmac" and cited an unrelated commit (the code says django-concat). The judge shows a control drop of -0.25 (llama2) and -0.12 (codellama) on 4 questions; intervals include 0.	More control questions; "no loss on control" is not claimed
Refusal gate does not transfer	Unanswerable but on-topic questions passed the relevance gate. codellama invented "Tristan Escalada paid for the library in 2014". llama2 abstained, but by itself, not via the gate.	Recalibrate G2 on several repositories
Model ignores or distorts evidence	Evidence retrieved (#111, #112) but codellama said 1.1.0 changed the default to HS512 (that was 1.0). starcoder2: 18 of 28 oracle answers hallucinated.	Instruction-tuned models
Models rarely cite the right source	Gold evidence cited in only 1-2% of answers; citation validity is high (0.9-1.0) because invented refs are stripped.	Stricter answer format
Automatic scoring is generous	143 full-credit answers; 42 rated lower by DeepSeek (wrong extra claim or a missing fact).	Stricter key facts; human grading

Limitations, assumptions and sources of bias

- **Sample size:** 24 history, 4 control and 7 guardrail questions on one real repository; 2 to 10 per category, so intervals are wide. The control category (4) cannot establish "no loss".
- **Residual circularity:** the demo repository and its questions were written together (so it only built and tuned the system). On the real repository the history is independent, but the questions and gold answers were drafted by the project team from the same threads, checked by script (every key fact occurs in the cited text) and by one independent review.
- **Independent review after freezing** found two weak items of 35 (it06: the gold says built-in json is "equivalent" to simplejson, which the thread disputes; it35: the history contains no real secrets, so "refuse" is ambiguous). They were not edited; results are reported with and without them (errata file in the repository).
- **Scoring:** automatic substring matching, validated against LLM judges (kappa 0.75 on 399 answers) and a hand check of 18 factual rows, but there is no human grading of the 16 technical rationale answers. Loose key-fact phrasings can give partial credit for echoing the question.
- **One repository, small local models** (6 GB GPU, 300-token answers, 3072-token context): results may differ for larger models or other settings.
- **Guardrail thresholds** (G1 0.36, G2 0.44) were calibrated on the demo dev split with only two negative examples each; G2 did not transfer.
- **Incident:** a first held-out attempt was lost when Docker Desktop stopped at about 70 of 399 answers; it was discarded unseen and rerun from scratch with no change to data or settings.

6. Methodology and reproducibility

Criterion: clarity of methodology, setup, datasets and configuration, and the ability to reproduce or independently understand the work.

Item	Value
Models (Ollama)	llama2:latest (78e26419b446), codellama:7b (8fdf8f752f6e), starcoder2:3b (9f4ae0aff61e); jury gemma:2b (b50d6c999e59)
Fixed settings	temperature 0, seed 42, max answer 300 tokens, context budget 6000 characters, top-k 8, G1 0.36, G2 0.44, G4 support 0.50; host Ollama with flash attention, 8-bit KV cache, context 3072
Held-out data	Snapshot of pallets/itsdangerous issues and PRs taken with the gh CLI (scripts/fetch_gh_fixture.py); indexed at commit 672971d; dataset hash b68e5bd36697c3ba, committed before the run (50bdc9e)
Dataset checks	scripts/check_dataset.py: 28 of 28 answerable items have existing evidence and every key fact present in it; errata for it06 and it35
Artefacts	reports/itsdangerous_heldout.md and .json (every stored answer), deepseek_judge.csv, gpt_grades.csv, gemini_grades.csv, human_grades_easy.csv, grading sheets
Documentation	docs/EVALUATION.md (full method and results), docs/VIVA_PREP.md, CODEORIGIN_CONTINUATION.md, README

Commands to reproduce the held-out result

```
cp .env.example .env
docker compose -f docker-compose.yml -f docker-compose.demo.yml -f docker-compose.ports.yml up -d --build
python scripts/fetch_gh_fixture.py pallets/itsdangerous demo-data/itsdangerous_fixtures.json
# index it: POST /api/repos with source=https://github.com/pallets/itsdangerous
# and issues_file=demo/itsdangerous_fixtures.json
python scripts/check_dataset.py services/eval/datasets/itsdangerous.json demo-data/itsdangerous_fixtures.json
python scripts/run_eval.py --live --dataset itsdangerous --repo pallets__itsdangerous --split test \
    --models llama2,codellama:7b,starcode2:3b --out reports/itsdangerous_heldout
python scripts/sensitivity.py reports/itsdangerous_heldout.json --exclude it06,it35
python scripts/judge_effect.py reports/deepseek_judge.csv
```

CI (11 jobs) runs the unit tests, drift checks and a mock-LLM end-to-end test for plumbing only; mock results are never used as evidence.

7. Individual understanding

Criterion: each student can explain and defend the question, methodology, implementation, experiments, results, limitations and conclusions.

Short answers to the questions an examiner is most likely to ask (longer versions in docs/VIVA_PREP.md):

Question	Answer
What is the research question and why does it matter?	Does repository history improve answers about why code was built a certain way? Assistants see only current code, so rationale questions become guesses.
Why four modes?	no_context = what the model guesses; code_only = realistic baseline; code_history = treatment; oracle = gold evidence with no retrieval, to tell retrieval failure from comprehension failure.
Why a pre-registered rule?	To avoid choosing a favourable reading after seeing results: at least +0.10, interval excluding 0, and no more than 0.05 lost on the control.
Why is starcoder2 "inconclusive"?	Its interval [-0.21, +0.17] includes 0 and its effect is about 0. The cause is the model: a 3B code-completion model that invents answers even when handed the gold evidence.
Why did codellama look "inconclusive" on the demo but pass on the real repo?	Same effect size, but only 13 questions on the demo gave a wide interval touching 0; 24 questions tighten it.
What is a bootstrap CI / permutation test?	Bootstrap: resample per-question differences with replacement; the middle 95% of resampled means is the interval. Permutation: if history made no difference the sign of each paired difference is arbitrary, so flip signs at random and see how often a difference this large occurs.
Is the test biased towards history?	On the demo, yes by construction. That is why the headline is a held-out real repository with independent history; some residual bias remains because we drafted the questions from its threads.

Question	Answer
Can you trust the scoring?	Partly. It is automatic key-fact matching, generous in about 29% of full-credit answers, but it agrees with a DeepSeek judge (kappa 0.75, 399 answers) and with a hand check of 18 factual rows; technical rows are not human-graded.
What went wrong?	Retrieval missed an unusual phrasing; the refusal gate did not fire for on-topic unanswerable questions; history distracted two models on code-only questions (4 questions).
What would you do next?	Human-grade the technical rows, add a second real repository with 40+ questions, fix retrieval and the refusal gate, and test a larger model.

Honesty statement. The result is promising, pre-registered and checked three ways, but small: one repository, 24 history questions, scoring that is only partly human-validated. We claim it is supported for 7B models on this repository, not that it is general. All numbers in this report come from the stored run files in the repository and can be recomputed with the scripts listed in Section 6.

Appendix. CI and CD runs, Prometheus, and the Grafana findings

This appendix answers three questions: what the CI and CD runs are and what they did, what Prometheus was used for, and everything the Grafana dashboards showed.

CI and CD runs

CI (continuous integration): on every push GitHub Actions builds the project and runs all checks automatically (lint and drift checks, six test jobs, frontend build, gitleaks and Trivy security scan, compose smoke test, end-to-end evaluation gate; 11 jobs, about 3 minutes). A red run means the change broke something. **CD (continuous delivery)**: when a version tag (v*) is pushed, the release workflow builds, scans and publishes the 7 container images to GHCR and creates a GitHub release. It is configured but has **not run yet**: no tag has been pushed, so nothing was published.

Run history (Code_Origin repository)	Count
Workflow runs so far	22
CI runs passed	14 (median 3.1 min, range 3.0 to 3.5)
CI runs failed	1: two lint errors in a new script, fixed in the next commit
CI runs cancelled	6: superseded by a newer push
Nightly evaluation runs	1, passed

Problems the runs caught: a HIGH vulnerability in a frontend dependency, Dockerfiles running as root, linter version drift, a wrong script argument in the smoke test, a wrong scanner version, and two lint errors in a new script.

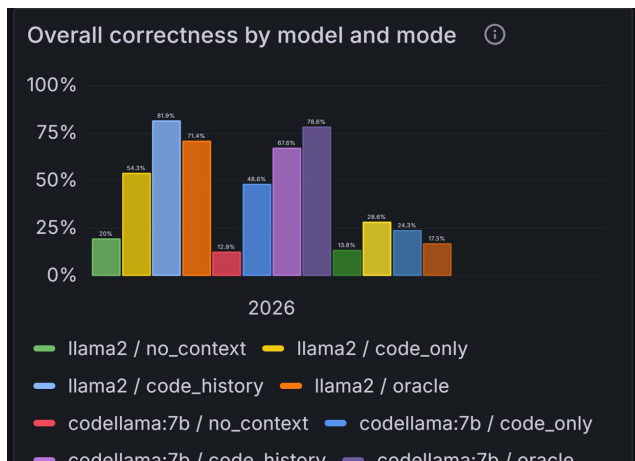
What Prometheus was used for

Prometheus is the metrics database behind the dashboard. It scrapes the gateway (every 15 s) and the evaluation service (every 30 s), stores the time series for 15 days, and evaluates three alert rules. It collects **16 metric families**: gateway (5: requests by route and status, request-time histogram, guardrail triggers, LLM tokens, refusals) and evaluation (11: correctness with its interval, history effect, verdict, failure taxonomy, jury score, run progress, quality metrics, latency, tokens). Alert rules: GatewayDown (unreachable 1 min), HighErrorRate (more than 5% 5xx over 5 min), SlowAsk (p95 of /ask above 60 s for 10 min); all healthy, none firing. Grafana queries Prometheus (and Loki for logs) to draw the dashboard.

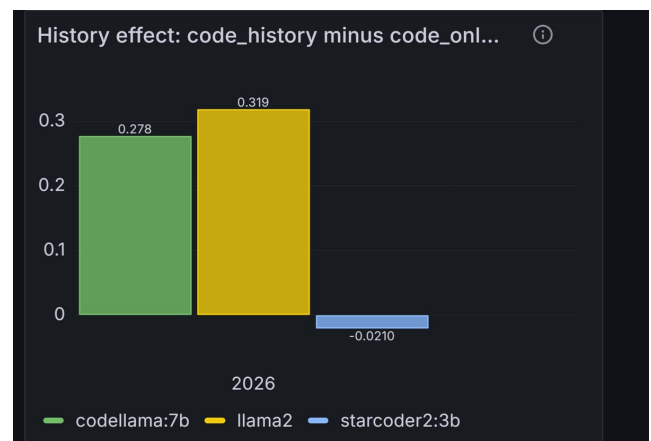
Grafana findings

Group	Finding
Evaluation (run #8)	Correctness no context / code only / code + history: llama2 20 / 54 / 82%, codellama 13 / 49 / 68%, starcoder2 14 / 29 / 24%. History effect +0.319, +0.278, -0.021: useful for the two 7B models, inconclusive for starcoder2. Retrieval misses fall from 11 / 10 / 16 (code only) to 3 / 2 / 6 (with history). The gemma:2b jury scored about 100%: too lenient.
Quality and cost	Hallucination rate 31%, 22%, 59%. Evidence recall 0.14, 0.70, 1.00 and precision 0.05, 0.25, 1.00 (code only, code + history, oracle). Mean latency 12.0 s, 14.5 s, 2.3 s; tokens written 126, 173, 147; unsupported-sentence ratio 27%, 26%, 37%. Refusal correctness 0.83 with no context, 0.89 to 1.00 with retrieval.
Live service metrics (5 demo questions)	All 5 /api/ask requests succeeded, no 5xx; p95 /ask latency about 26 s; 3 answers used 6,635 tokens in and 554 out; guardrails G1 (off-topic), G10 (unsafe request) and G4 (grounding flag) fired; no alert firing.

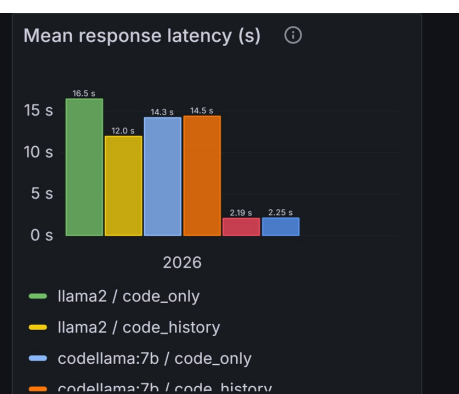
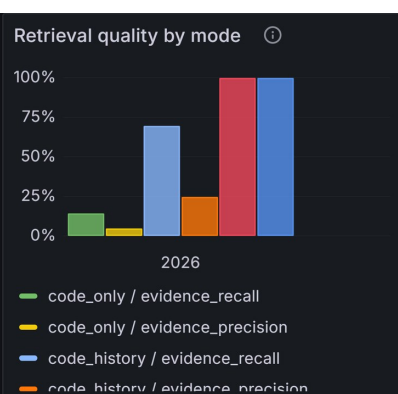
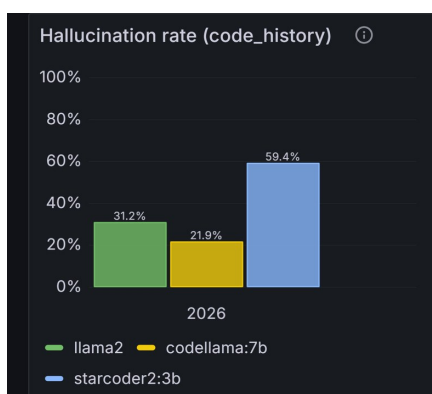
Dashboard panels (Grafana, run #8)



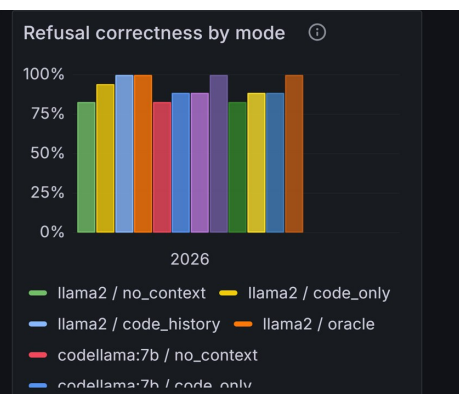
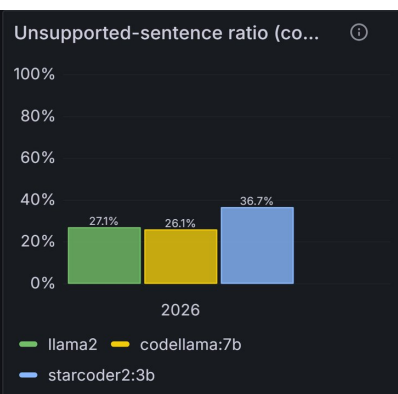
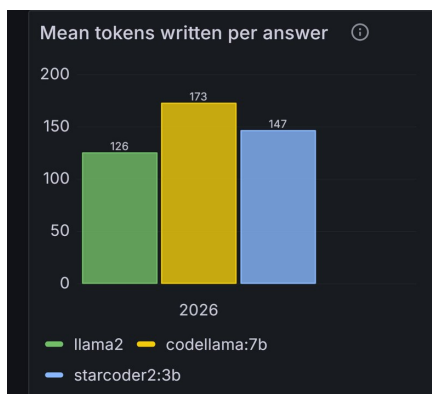
Overall correctness by model and mode



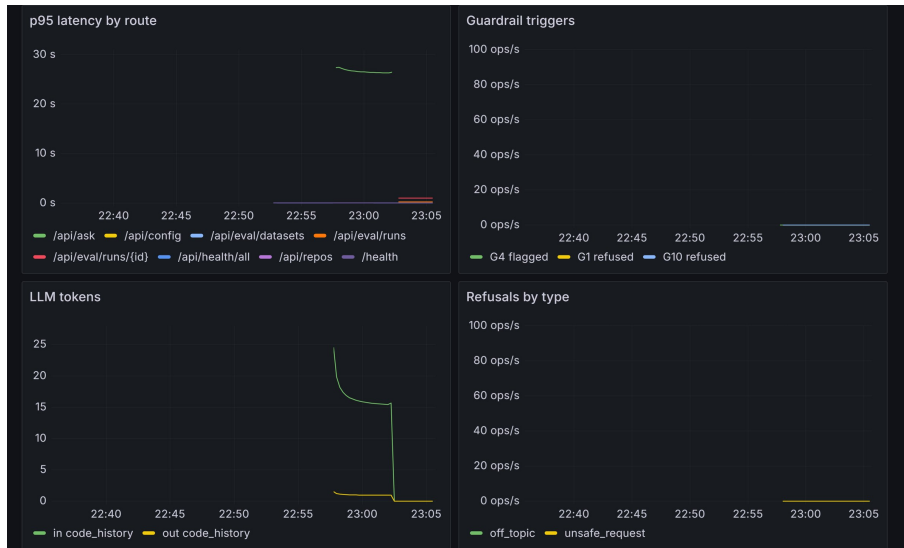
History effect: code_history minus code_onl...



Hallucination rate, retrieval quality by mode, mean response latency



Mean tokens per answer, unsupported-sentence ratio, refusal correctness by mode



Live service panels: p95 latency by route, guardrail triggers, LLM tokens, refusals by type (five demo questions)